How to Actually Understand This Project

A 5-session, ~4-hour study plan that gets you from "I read the docs" to "I can defend any decision confidently." Built on the principle that you own something only when you can rebuild it in your head without looking.

Contents

The core principle

> **You understand something when you can explain it in your own words without looking.** Every time you finish a section, close the file and try to explain it to yourself out loud. If you stumble, you don't get it yet — go back.

This is the **Feynman technique**, and it's what separates people who *read* code from people who *own* it. Interviewers can tell within 30 seconds which one you are.

Session 1 — Trace one request end-to-end (45 min)

Pick one thing: **a `POST /logs` request**. Follow it through every layer, taking notes *on paper* (not typing — writing forces you to synthesize).

Order of files to open

1. `src/server.ts` — What's the very first thing that happens when Node starts? (Load config → connect to Postgres → run migrations → start Fastify → start retention worker → flip `ready = true`).
2. `src/app.ts` — How does Fastify know about the `/logs` route? (`app.register(ingestRoutes(...))`).

3. `src/http/logsIngest.ts` — What checks happen before we touch the DB? (top-level shape → batch size → per-entry validation).
4. `src/domain/logSchemas.ts` — What does `validateLogEntry` actually check? Read it line by line.
5. `src/db/repositories/logRepository.ts` — Look at `insertMany`. Understand why we build 5 parallel arrays and pass them to `UNNEST`.
6. `migrations/001_init.sql` — Understand what the row lands in.

The exercise

After reading, close everything. On paper, draw the flow: `HTTP request → ??? → ??? → ??? → HTTP response`. Fill in every step. When done, reopen the files and check what you missed. Do this **3 times** until the diagram is complete from memory.

> **Confidence check:** Can you answer "walk me through a POST /logs request" for a full minute without looking? If yes, move on to Session 2.

Session 2 — Speak to Postgres directly (45 min)

The database is the interesting part of this project. If you can converse with `psql`, you can defend any schema question.

Bring the stack up:

```
docker compose up -d
```

Open a psql session:

```
docker compose exec postgres psql -U logs -d logs
```

Run these commands one at a time and read every output carefully:

```
-- 1. See the parent table and its partitions
\d+ logs

-- 2. List all child partitions (partitioning made real)
SELECT inhrelid::regclass FROM pg_inherits WHERE inhparent='logs'::regclass;

-- 3. List all indexes
\di

-- 4. Look at one child partition and its indexes
\d+ logs_p_20260808   -- adjust date

-- 5. Insert one row manually so you understand the ENUM cast
INSERT INTO logs (timestamp, level, service, message, attributes)
VALUES (NOW(), 'error'::log_level, 'demo', 'hi', '{"user_id":"1"}'::jsonb);

-- 6. Try to break it -- invalid level
INSERT INTO logs (timestamp, level, service, message)
VALUES (NOW(), 'critical'::log_level, 'demo', 'hi');
-- ^ this fails at the DB layer because 'critical' is not in the enum

-- 7. Query with EXPLAIN to see which index is used
EXPLAIN ANALYZE
SELECT * FROM logs WHERE service = 'demo' ORDER BY timestamp DESC LIMIT 10;

-- 8. Now drop that index and re-run to see the difference
BEGIN;
DROP INDEX idx_logs_svc_ts_id;
EXPLAIN ANALYZE
SELECT * FROM logs WHERE service = 'demo' ORDER BY timestamp DESC LIMIT 10;
ROLLBACK;

-- 9. See the JSONB containment operator in action
SELECT * FROM logs WHERE attributes @> '{"user_id":"1"}'::jsonb;

-- 10. See a bucket aggregation query
SELECT date_bin(INTERVAL '1 minute', timestamp, TIMESTAMPTZ 'epoch') AS b,
       service, count(*)
FROM logs
WHERE timestamp >= NOW() - INTERVAL '1 hour'
GROUP BY b, service
ORDER BY b;
```

The exercise

After each `EXPLAIN`, explain out loud what plan Postgres chose and why. Look for keywords: `Index Scan`, `Seq Scan`, `Bitmap Heap Scan`, `HashAggregate`, `Sort`.

> **Confidence check:** Can you say why `EXPLAIN` for step 7 was fast, and why step 8 became slower? If yes, you understand what indexes actually do.

Session 3 — Break things on purpose (45 min)

You learn 10× more from a broken system than a working one. Do these in order.

Break 1 — comment out one index and re-measure

Edit `migrations/002_indexes.sql` and comment out `idx_logs_ts_id`. Then:

```
docker compose down -v
docker compose up -d --build
```

Ingest ~100k rows via the smoke script, then run the aggregate `EXPLAIN ANALYZE`. Notice what Postgres does now. Uncomment the index and repeat.

> **Question to answer:** Which index does Postgres actually pick for what?

Break 2 — introduce a validation bug

In `src/domain/logSchemas.ts`, change the level check from:

```
if (typeof entry.level !== "string" || !LEVEL_SET.has(entry.level)) {
```

to:

```
if (typeof entry.level !== "string") {
```

Run the tests: `npm test`. Watch which tests fail. Read the error messages. Revert.

> **Question to answer:** Which specific test caught this and why?

Break 3 — watch the wrong sort behavior

In `src/db/repositories/logRepository.ts`, find:

```
ORDER BY "timestamp" DESC, id DESC
```

Change it to:

```
ORDER BY "timestamp" DESC
```

Run the integration test that tests deterministic pagination with same-timestamp rows. Watch it fail. Understand exactly why. Revert.

> **Question to answer:** Why is `id` in the ORDER BY not just decoration — what breaks without it?

Break 4 — watch retention fail safely

In `src/retention/retentionWorker.ts`, throw an error in `dropExpiredPartitions`:

```
private async dropExpiredPartitions(): Promise<void> {
  throw new Error("simulated failure");
}
```

Restart the app. Watch the logs. Notice the app keeps running — retention failures don't crash ingest. Revert.

> **Question to answer:** Why did I isolate retention from the ingest path?

Session 4 — The foundational concepts (60 min)

For each of these, read ~10 minutes on the web AND explain it out loud in one sentence.

Concept	What to search	One-sentence answer to memorize
Partitioning	"Postgres declarative partitioning"	Splitting one logical table into physical child tables by a range or list key.
Index Only Scan	"Postgres index only scan visibility map"	An index scan that never touches the heap because the index has all needed columns AND the visibility map says the pages are all-visible.
GIN index	"Postgres GIN index jsonb_path_ops"	A generalized inverted index — one row can have many indexable elements (like all keys in a JSONB).
Trigram	"Postgres pg_trgm gin_trgm_ops"	Splitting text into 3-character sequences so you can index substring search.
WAL	"Postgres write ahead log synchronous_commit"	Every change is first appended to a log for durability; `synchronous_commit=off` means the commit returns before the log is fsync'd.
Autovacuum	"Postgres autovacuum visibility map"	Background process that reclaims dead-tuple space AND marks pages "all visible" so index-only scans can skip the heap.
Cursor pagination	"keyset pagination vs offset"	Encoding the last row's sort keys in the cursor so `WHERE (ts, id) < (last_ts, last_id)` jumps straight to the next page via the index.
BitmapAnd	"Postgres bitmap heap scan bitmapand"	Combines two indexes: build a bitmap of matching rows from each, AND them, then fetch the heap rows once. Why multi-filter queries stay fast.
date_bin	"Postgres date_bin function"	Rounds a timestamp down to a fixed interval anchored on an origin — the modern replacement for `date_trunc` with irregular intervals.
Connection pooling	"pg node pool max"	Pre-open N Postgres connections and reuse them so each HTTP request doesn't pay the TCP + auth handshake cost.

The exercise

For each concept, write your one-sentence answer **on paper without looking**. Then check it against the table. Any concept you miss goes back on your list for tomorrow.

Session 5 — Mock interview (45 min)

Simulate the real thing. Set a timer. Answer each question **out loud** as if someone were listening. Record yourself on your phone if you can, then play it back — you'll notice every "um" and every place you hedge.

#	Question	Time
Q1	What is this project?	1 min
Q2	Walk me through what happens when a client POSTs a batch of 500 logs.	2 min (full end-to-end)
Q3	Why did you partition by day?	1 min
Q4	Why is `id` in your ORDER BY?	30 sec
Q5	I'm looking at your GIN index — what does `jsonb_path_ops` mean and why not the default?	1 min
Q6	How would `EXPLAIN ANALYZE` on your aggregation query look?	1 min (describe the plan without running it)
Q7	Where's the SQL injection risk in this code?	30 sec (trick question — nowhere, and here's why)
Q8	You're at 20k logs/s peak but aggregate p95 is 1.5s under mixed load. Defend that.	2 min (honest, own the trade-off, propose the rollup fix)
Q9	Why Fastify not Express?	30 sec
Q10	What would you do differently if you had another week?	1 min
Q11	What's the worst thing that could happen to this system in production?	1 min (think through failure modes)
Q12	A user says the last hour of logs is missing. Where do you look?	1 min (check ingested_at vs timestamp, retention logs, default partition)

How to grade yourself

- **Answered clearly and confidently in under the time limit** → understood.
- **Rambled or hedged** → open the relevant file, re-read, retry the question.
- **Didn't know** → back to Session 1 or 2 for that area.

The single best thing you can do

> **Explain the project to a friend or a rubber duck for 15 minutes straight without looking at anything.** If you can hold a monologue about the schema, indexes, and trade-offs for a full quarter hour without stumbling, you're interview-ready. If you can't, you've found the gap you need to close.

The interviewer isn't looking for a memorized script — they're looking for someone who clearly understands the trade-offs they made. If you can say "*I chose X because Y, and I considered Z but rejected it because W*" for every design decision, you'll pass.

Your 3-day schedule

Day	Sessions	Total time
Day 1	Session 1 (trace request) + Session 4 (concepts)	~1h 45m
Day 2	Session 2 (psql) + Session 3 (break things)	~1h 30m
Day 3	Session 5 (mock interview) + read the interview-guide PDF once more	~1h 30m

Between sessions, keep `docs/interview-guide.pdf` open on your phone. Read it in bed, on the bus, during coffee. Repetition is what turns knowledge into confidence.

Companion documents: `docs/interview-guide.pdf` (deep-dive reference) · `docs/demo-video-script.md` (5-minute demo teleprompter).